

Отчёт по задаче «Спутники и группировки»

Смола Артём, группа 23.Б08

1 Постановка задачи

Даны n космических аппаратов (КА) и m взаимодействий между ними. Для каждого взаимодействия задан интервал времени на общей шкале $[0, T)$. Требуется разбить глобальный интервал на такты по 10 минут и для каждого такта получить группировки КА: два КА относятся к одной группировке, если в данном такте они связаны цепочкой активных взаимодействий.

2 Принцип решения

Для каждого такта $[t, t + 10)$ строится граф активных связей: ребро между КА активно, если его интервал взаимодействия пересекается с текущим тактом. Все интервалы трактуются как полуинтервалы вида $[start, finish)$. Если во входе задано поле `len`, дополнительно проверяется равенство `len = finish - start`.

Группировки в такте вычисляются как компоненты связности графа. Для поиска компонент используется DSU (система непересекающихся множеств):

1. Инициализировать DSU из n вершин.
2. Для каждого активного ребра выполнить объединение его концов.
3. После обработки всех рёбер собрать вершины по корням DSU.

Процедура повторяется для всех тактов на интервале $[0, T)$ с шагом 10 минут.

3 Корректность

На каждом такте строится граф активных связей. В него попадают ровно те рёбра, чьи интервалы пересекаются с текущим тактом.

После обработки активных рёбер DSU точно задаёт компоненты связности. По определению задачи каждая такая компонента является группировкой КА. Так как процедура выполняется для всех тактов, программа выдаёт корректный ответ для всей шкалы времени.

4 Ограничения реализации

- единая семантика времени: только полуинтервалы $[start, finish)$;
- требуется `start < finish` для каждой связи;
- в формате с `len` требуется строго `len = finish - start`;
- если задано T , то для каждой связи должно выполняться `finish ≤ T`.

5 Исходный код

Исходный код программы и тестовые данные доступны в репозитории GitHub: https://github.com/artem-smola/formal_lang.

6 Код программы

Код программы:

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <map>
#include <numeric>
#include <sstream>
#include <string>
#include <vector>

struct Interaction {
    int u;
    int v;
    int start;
    int finish; // Полуинтервал [start, finish)
};

class DSU {
public:
    explicit DSU(int n) : parent_(n + 1), size_(n + 1, 1) {
        std::iota(parent_.begin(), parent_.end(), 0);
    }

    int find(int x) {
        if (parent_[x] == x) {
            return x;
        }
        parent_[x] = find(parent_[x]);
        return parent_[x];
    }

    void unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) {
            return;
        }
        if (size_[a] < size_[b]) {
            std::swap(a, b);
        }
        parent_[b] = a;
        size_[a] += size_[b];
    }

private:
    std::vector<int> parent_;
    std::vector<int> size_;
};

static std::string trim(std::string s) {
    while (!s.empty() && std::isspace(static_cast<unsigned char>(s.front())))) {
```

```

        s.erase(s.begin());
    }
    while (!s.empty() && std::isspace(static_cast<unsigned char>(s.back()))) {
        s.pop_back();
    }
    return s;
}

static std::vector<int> parse_ints_from_line(const std::string& line) {
    std::vector<int> values;
    std::istringstream iss(line);
    int value = 0;
    while (iss >> value) {
        values.push_back(value);
    }
    return values;
}

static bool intervals_intersect_half_open(int a1, int b1, int a2, int b2) {
    return std::max(a1, a2) < std::min(b1, b2);
}

static std::vector<std::vector<int>> groups_for_tick(int n, const
    ↪ std::vector<Interaction>& edges,
                                     int tick_start, int
    ↪ tick_finish) {
    DSU dsu(n);
    for (const auto& e : edges) {
        if (intervals_intersect_half_open(e.start, e.finish, tick_start,
            ↪ tick_finish)) {
            dsu.unite(e.u, e.v);
        }
    }

    std::map<int, std::vector<int>> groups_map;
    for (int ka = 1; ka <= n; ++ka) {
        groups_map[dsu.find(ka)].push_back(ka);
    }

    std::vector<std::vector<int>> groups;
    groups.reserve(groups_map.size());
    for (auto& [root, members] : groups_map) {
        (void)root;
        groups.push_back(std::move(members));
    }

    std::sort(groups.begin(), groups.end(), [](const std::vector<int>& a, const
    ↪ std::vector<int>& b) {
        return a.front() < b.front();
    });
    return groups;
}

int main(int argc, char** argv) {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    bool machine_mode = false;
    if (argc > 1) {
        const std::string arg = argv[1];

```

```

        if (arg == "--machine") {
            machine_mode = true;
        } else {
            std::cerr << "Ошибка: поддерживается только опция --machine.\n";
            return 1;
        }
    }

    std::vector<std::string> lines;
    std::string raw;
    while (std::getline(std::cin, raw)) {
        const std::string t = trim(raw);
        if (t.empty() || t[0] == '#') {
            continue;
        }
        lines.push_back(t);
    }

    if (lines.empty()) {
        std::cerr << "Ошибка: пустой ввод.\n";
        return 1;
    }

    const std::vector<int> header = parse_ints_from_line(lines[0]);
    if (header.size() != 2 && header.size() != 3) {
        std::cerr << "Ошибка: первая строка должна содержать `n m` или `n m`  

        ↪ `T`.\n";
        return 1;
    }

    const int n = header[0];
    const int m = header[1];
    int global_t = (header.size() == 3 ? header[2] : -1);

    if (n <= 0 || m < 0) {
        std::cerr << "Ошибка: некорректные n или m.\n";
        return 1;
    }

    if (header.size() == 3 && global_t < 0) {
        std::cerr << "Ошибка: T должно быть неотрицательным.\n";
        return 1;
    }

    if (static_cast<int>(lines.size()) < 1 + m) {
        std::cerr << "Ошибка: ожидалось " << m << " строк со связями.\n";
        return 1;
    }

    std::vector<Interaction> edges;
    edges.reserve(m);

    int max_finish = 0;
    for (int idx = 0; idx < m; ++idx) {
        const std::vector<int> v = parse_ints_from_line(lines[1 + idx]);
        if (v.size() != 4 && v.size() != 5) {
            std::cerr << "Ошибка: строка связи должна содержать 4 или 5 целых  

            ↪ чисел.\n";
            return 1;
        }
    }

```

```

const int u = v[0];
const int w = v[1];
int start = 0;
int finish = 0;
if (v.size() == 4) {
    start = v[2];
    finish = v[3];
} else {
    const int len = v[2];
    start = v[3];
    finish = v[4];
    if (len <= 0) {
        std::cerr << "Ошибка: len должен быть положительным.\n";
        return 1;
    }
    if (finish - start != len) {
        std::cerr << "Ошибка: ожидается len = finish - start для
        ↪ полуинтервала [start, finish).\n";
        return 1;
    }
}

if (u < 1 || u > n || w < 1 || w > n || u == w) {
    std::cerr << "Ошибка: номера КА должны быть в диапазоне 1..n и не
    ↪ совпадать.\n";
    return 1;
}
if (start < 0 || finish < 0 || start >= finish) {
    std::cerr << "Ошибка: требуется строгий полуинтервал [start,
    ↪ finish), где start < finish.\n";
    return 1;
}

edges.push_back({u, w, start, finish});
max_finish = std::max(max_finish, finish);
}

if (global_t < 0) {
    global_t = max_finish;
}
if (global_t < 0) {
    std::cerr << "Ошибка: не удалось определить T.\n";
    return 1;
}

for (const auto& e : edges) {
    if (e.finish > global_t) {
        std::cerr << "Ошибка: интервал связи выходит за пределы [0, T).\n";
        return 1;
    }
}

constexpr int kTick = 10;
if (global_t == 0) {
    if (machine_mode) {
        std::cout << "T 0\n";
    } else {
        std::cout << "Глобальный интервал [0, 0). Тактов для обработки
        ↪ нет.\n";
    }
}

```

```

        return 0;
    }

    for (int tick_start = 0; tick_start < global_t; tick_start += kTick) {
        const int tick_finish = std::min(tick_start + kTick, global_t);
        const std::vector<std::vector<int>> groups = groups_for_tick(n, edges,
            ↪ tick_start, tick_finish);

        if (machine_mode) {
            std::cout << "tick " << tick_start << " " << tick_finish << " groups
            ↪ " << groups.size() << "\n";
            for (size_t i = 0; i < groups.size(); ++i) {
                std::cout << "group " << (i + 1) << " " << groups[i].size();
                for (int ka : groups[i]) {
                    std::cout << " " << ka;
                }
                std::cout << "\n";
            }
            continue;
        }

        std::cout << "Такт [" << tick_start << ", " << tick_finish << "):\n";
        for (size_t i = 0; i < groups.size(); ++i) {
            std::cout << " Группировка " << (i + 1) << ":";
            for (int ka : groups[i]) {
                std::cout << ' ' << ka;
            }
            std::cout << '\n';
        }
        std::cout << '\n';
    }

    return 0;
}

```

7 Демонстрация работы программы

1) Входные данные

Рассматриваемый тест: **test1**.

Формат входных данных:

1. первая строка: n, m или n, m, T ;
2. далее m строк связи:
 - $i\ j\ start\ finish$, или
 - $i\ j\ len\ start\ finish$;
3. такт фиксирован: 10 минут.

Где i, j — номера КА, а интервал взаимодействия всегда задаётся как полуинтервал $[start, finish)$. Если T не задано, программа берёт $T = \max(finish)$ по входным связям.

```

5 4 30
1 2 10 0 10
2 3 10 8 18
4 5 30 0 30
3 4 10 20 30

```

2) Вывод

```
• artem@artem-ВМН-WCX9:~/ТП_6_семестр/ТФЯИТ/Задания/KA_Groupings/Solution/Code$ ./ka_groups < tests/test1.in
Такт [0, 10):
  Группировка 1: 1 2 3
  Группировка 2: 4 5

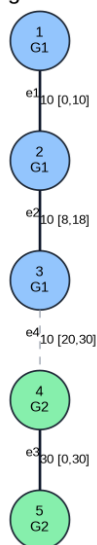
Такт [10, 20):
  Группировка 1: 1
  Группировка 2: 2 3
  Группировка 3: 4 5

Такт [20, 30):
  Группировка 1: 1
  Группировка 2: 2
  Группировка 3: 3 4 5
```

3) Визуализация

Такт [0, 10)

KA Groupings: test1; tick [0, 10)



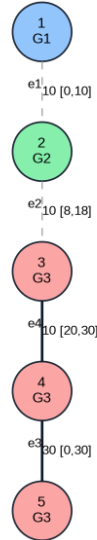
Такт [10, 20)

KA Groupings: test1; tick [10, 20)



Такт [20, 30)

KA Groupings: test1; tick [20, 30)



4) Описание результата

Для теста **test1**:

- в такте $[0, 10)$ формируются группы $\{1, 2, 3\}$ и $\{4, 5\}$;
- в такте $[10, 20)$ формируются группы $\{1\}$, $\{2, 3\}$ и $\{4, 5\}$;
- в такте $[20, 30)$ формируются группы $\{1\}$, $\{2\}$ и $\{3, 4, 5\}$.

Изменение группировок по тактам соответствует активации и деактивации интервалов взаимодействия.

5) Дополнительный граничный тест

Для **test2** используется формат без явного T (оно определяется как максимум finish), а также несколько пересечений на границах тактов.

Входные данные теста test2:

```
6 5
1 2 0 9
2 3 10 20
3 4 15 25
4 5 0 30
5 6 5 15
```

Результат отражает полуинтервальную семантику. В такте $[10, 20)$ формируется группировка $\{2, 3, 4, 5, 6\}$, а КА 1 остаётся изолированным.

8 Анализ сложности

Пусть $q = \lceil T/10 \rceil$ — число тактов.

Для каждого такта:

- проверяются все m связей на активность;
- выполняются операции DSU почти за константу (амортизированно).

Итоговая временная сложность:

$$O(q \cdot (m + n)).$$

Память:

$$O(n + m)$$

на хранение связей и структур DSU.